

CSI 3140 Lab 4 Report: Client-Server Interaction and Modern Web Services

Course Information

Course Code	CSI 3140
Lab Number	Lab 4
Project	Uranus Fitness Registrations API
Team Members	David Gvozdyev - 300308910 Liam Geraghty - 300356748 Neelman Bhardwaj - 300389998
Date Submitted	2026-07-11

1. Project Overview

1.1 Topic Description

Uranus Fitness is a community-focused fitness training website that allows visitors to review available training plans, compare their duration and price, and submit a membership registration. Laboratory 4 extends the interactive website created in earlier laboratories by adding a Node.js and Express.js server, a REST API, JSON data storage, OpenAPI documentation, and browser-to-server communication using `fetch()`.

1.2 Laboratory 4 Goals

- Serve the existing HTML, CSS, images, and JavaScript from an Express server.
- Expose plan and registration resources through meaningful REST endpoints.
- Send and receive JSON using appropriate HTTP methods and status codes.
- Load plan data dynamically on the schedule page using `fetch()`.
- Submit registration form data to the server using a POST request.
- Validate requests on both the client and server and return safe error messages.

- Document and test the API using OpenAPI/Swagger and an API testing tool.
- Compare REST/OpenAPI with SOAP/WSDL, GraphQL, and gRPC.

1.3 Client-Server Architecture

```

Browser client (public/ HTML, CSS, JavaScript)
  |
  | HTTP requests using fetch()
  v
Node.js + Express server (server.js)
  |
  | Routes and validation
  v
Express routers (routes/plans.js, routes/registrations.js)
  |
  | Data access functions
  v
dataStore.js -> data/plans.json and data/registrations.json

```

The browser and API are served from the same Express application. This keeps the project simple and avoids cross-origin configuration during local development. The front end is responsible for user interaction and page updates, while the server is responsible for resources, validation, persistence, status codes, and JSON responses.

2. Project Structure

2.1 Directory Layout

```

Lab4/
|-- public/
|   |-- index.html
|   |-- about.html
|   |-- schedule.html

```

```

| |-- registration.html
| |-- styles.css
| |-- script.js
| |-- index.js
| |-- about.js
| |-- schedule.js
| |-- registration.js
| `-- img/
|     |-- logo.png
|     `-- old_guypng.png
|-- routes/
|   |-- plans.js
|   `-- registrations.js
|-- data/
|   |-- plans.json
|   `-- registrations.json
|-- api-docs/
|   `-- openapi.yaml
|-- screenshots/
|-- dataStore.js
|-- server.js
|-- package.json
|-- package-lock.json
|-- README.md
`-- report.pdf

```

2.2 Main Files and Responsibilities

File	Responsibility
server.js	Creates the Express app, enables JSON parsing, serves public files, mounts routers, exposes /api/status, and serves Swagger UI at /api-docs.
routes/plans.js	Handles plan resource requests such as listing all plans and retrieving one plan by ID.

File	Responsibility
routes/registrations.js	Handles registration CRUD requests, validation, status codes, and persistence.
dataStore.js	Reads JSON files, finds records, calculates the next registration ID, and writes registration changes to disk.
data/plans.json	Read-only source of plan records used by the schedule page and API.
data/registrations.json	Local persistence file for registration records.
public/schedule.js	Fetches plans from the API, renders table rows, filters, sorts, shows details, and stores the selected plan in localStorage.
public/registration.js	Validates the form, builds a JSON request body, submits it to the API, and displays success or error feedback.
api-docs/openapi.yaml	Defines API metadata, endpoints, request bodies, response schemas, examples, and status codes.
package.json	Defines the npm start command and dependencies: Express, Swagger UI Express, and YAMLJS.

3. Installation and Execution

3.1 Requirements

- Node.js 18 or newer is recommended.
- npm, included with Node.js.
- A modern browser such as Chrome, Edge, or Firefox.
- Postman, Thunder Client, curl, or Swagger UI for API testing.

3.2 Running the Application

1. Open a terminal in the Lab4 project folder.
2. Run npm install to install the dependencies from package.json.
3. Run npm start. The start script executes node server.js.
4. Open <http://localhost:3000> in a browser.

5. Open `http://localhost:3000/api-docs` to view and test the OpenAPI documentation through Swagger UI.
6. Stop the server by pressing `Ctrl+C` in the terminal.

```
npm install
npm start

# Front end
http://localhost:3000

# Swagger UI
http://localhost:3000/api-docs
```

3.3 Express Server Setup

The server listens on port 3000. `express.json()` parses incoming JSON request bodies, while `express.static("public")` exposes the client files. The plan and registration routers are mounted under resource-based paths. Swagger UI reads the OpenAPI YAML document and presents interactive documentation.

```
app.use(express.json());
app.use(express.static("public"));
app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(openApiDocument));
app.use("/api/plans", plansRouter);
app.use("/api/registrations", registrationsRouter);
```

4. REST API Design

4.1 Resource Design

The API uses nouns rather than action verbs. Plans and registrations are modeled as resources. Collection routes operate on all records, while routes containing `:id` operate on one specific record. JSON is used for request and response bodies.

4.2 Endpoint Documentation

Method	Endpoint	Purpose	Success	Errors
GET	/api/status	Check whether the API is running.	200	-
GET	/api/plans	Return all training plans.	200	-
GET	/api/plans/:id	Return one plan by numeric ID.	200	404
GET	/api/registrations	Return all registrations.	200	-
POST	/api/registrations	Create a new registration.	201	400
GET	/api/registrations/:id	Return one registration by ID.	200	404
PUT	/api/registrations/:id	Replace all editable registration fields.	200	400, 404
PATCH	/api/registrations/:id	Update selected registration fields.	200	400, 404
DELETE	/api/registrations/:id	Delete one registration.	200	404

4.3 HTTP Methods and Status Codes

- GET retrieves data without modifying it.
- POST creates a new registration and returns 201 Created.
- PUT replaces a complete registration record.
- PATCH changes only the fields supplied by the client.
- DELETE removes a registration.
- 200 OK indicates a successful read, update, or deletion.
- 400 Bad Request indicates missing or invalid input.
- 404 Not Found indicates that the requested plan or registration does not exist.

5. JSON Data and Persistence

5.1 Plan Data

plans.json contains the Beginner, Intermediate, and Advanced plans. Each object includes an ID, name, duration, style, price, level, and description. The

schedule page does not need to duplicate these records in HTML; it receives them from the API and creates table rows dynamically.

```
{
  "id": 1,
  "name": "Beginner Plan",
  "duration": "2 weeks",
  "style": "Machine-assisted exercises",
  "price": 49,
  "level": "beginner",
  "description": "Best for new members who want guided practice and safer
pacing."
}
```

5.2 Registration Data

registrations.json stores registration records. The file contains at least four starting records, satisfying the laboratory requirement for four data objects with four or more fields. Each registration is linked to a plan through planId.

5.3 Data Store Module

dataStore.js separates file access from routing logic. It loads both JSON files when the server starts. Plan records are read through helper functions. Registration changes are saved using fs.writeFileSync(), which is sufficient for a small local laboratory project but would be replaced by a database in production.

- getPlans() and getRegistrations() return collections.
- findPlanById() and findRegistrationById() return individual resources.
- findPlanByLevel() maps the form tier to a plan and planId.
- getNextRegistrationId() calculates a new unique numeric ID.
- saveRegistrations() writes the current registration array to registrations.json.

6. JSON Request and Response Examples

6.1 Successful GET Response

GET /api/plans/1

HTTP/1.1 200 OK

Content-Type: application/json

```
{  
  "id": 1,  
  "name": "Beginner Plan",  
  "duration": "2 weeks",  
  "style": "Machine-assisted exercises",  
  "price": 49,  
  "level": "beginner",  
  "description": "Best for new members who want guided practice and safer  
pacing."  
}
```

6.2 Successful POST Request

POST /api/registrations

Content-Type: application/json

```
{  
  "id": 5,  
  "fullName": "Neelmani Bhardwaj",  
  "email": "bhardwaj009@gmail.com",  
  "phone": "2345672234",  
  "ageRange": "18-25",  
  "tier": "beginner",  
  "planId": 1,  
  "contactMethod": "email",  
}
```



```
"startDate": "2026-07-25",  
"about": "hello"  
}
```

6.3 Successful POST Response

HTTP/1.1 201 Created

Content-Type: application/json

```
{  
  "id": 5,  
  "fullName": "Neelmani Bhardwaj",  
  "email": "bhardwaj009@gmail.com",  
  "phone": "2345672234",  
  "ageRange": "18-25",  
  "tier": "beginner",  
  "planId": 1,  
  "contactMethod": "email",  
  "startDate": "2026-07-25",  
  "about": "hello"  
}
```

6.4 Error Responses

HTTP/1.1 400 Bad Request

```
{  
  "error": "fullName, email, phone, ageRange, tier, contactMethod,  
startDate, and about are required."  
}
```

```
HTTP/1.1 404 Not Found
{
  "error": "Registration not found."
}
```

7. Front-End and Back-End Interaction

7.1 Schedule Page Flow

1. The browser loads `schedule.html`, `styles.css`, `script.js`, and `schedule.js` from Express static hosting.
2. `schedule.js` waits for `DOMContentLoaded` and creates the filter, sort, feedback, and plan-details controls.
3. `loadPlans()` sends `GET /api/plans` using `fetch()`.
4. The API returns a JSON array of plans.
5. `response.json()` converts the JSON response into JavaScript objects.
6. `renderPlans()` creates a table row for every visible plan.
7. Filtering and sorting operate on the downloaded plans array without making unnecessary new API requests.
8. Selecting Register stores the plan level in `localStorage` so `registration.html` can preselect the matching radio button.

7.2 Registration Page Flow

1. The user completes the registration form.
2. `registration.js` performs client-side validation and shows inline errors.
3. A payload object is created from the form values.
4. `JSON.stringify()` converts the object into JSON text.
5. `fetch()` sends `POST /api/registrations` with `Content-Type: application/json`.
6. `express.json()` parses the request body on the server.
7. The registration router repeats validation, finds the selected plan, assigns `planId` and a new ID, and saves the updated JSON data.

8. The API returns either 201 Created with the new record or a safe 400 error response.
9. The front end resets the form and shows a success message only when `response.ok` is true.

7.3 Why `schedule.js` Performs Two Roles

`schedule.js` both communicates with the API and manages the page interface. This is reasonable for a small project. The API supplies the data, while browser JavaScript renders and interacts with that data. The plan records should exist only in `plans.json`; the HTML should keep the table structure but not duplicate the plan rows. In a larger application, API access and view rendering could be moved into separate JavaScript modules.

8. Validation and Error Handling

8.1 Client-Side Validation

Field	Client Rule
Full name	Required; whitespace-only values are rejected.
Email	Required and checked against a basic email pattern.
Phone	Required and checked against the accepted phone pattern.
Age range	A non-default option must be selected.
Tier	One of the membership radio buttons must be selected.
Contact method	A contact option must be selected.
Start date	A date must be provided.
Goals/About	A non-empty explanation must be provided.

8.2 Server-Side Validation

Server validation is required even when the browser validates first, because clients can disable JavaScript or call the API directly. Registration creation and complete replacement require `fullName`, `email`, `phone`, `ageRange`, `tier`, `contactMethod`, `startDate`, and `about`. The tier must match an available plan level. Invalid input receives 400 Bad Request. Missing IDs receive 404 Not Found.

8.3 Safe Error Handling

- The API returns concise JSON error objects rather than raw stack traces.
 - The schedule page checks `response.ok` and displays a readable loading failure message.
 - The registration page displays server-provided validation messages when available.
 - Network failures are caught and displayed as an inability to reach the server.
 - The user interface clears outdated messages before each new validation attempt.
-

9. OpenAPI and Swagger Documentation

The project includes an OpenAPI 3.0.3 document with the title Uranus Fitness Registrations API and base URL `http://localhost:3000`. The document defines plan and registration schemas, required input fields, path parameters, example requests and responses, and possible status codes. `server.js` loads this YAML file through `YAMLJS` and serves an interactive Swagger UI at `/api-docs`.

9.1 OpenAPI as an API Contract

OpenAPI acts as a machine-readable contract for a REST API. It tells developers and testing tools which endpoints exist, which data shapes are accepted, and which responses may be returned. The implementation and OpenAPI file must remain synchronized. This role is similar to the contract role that WSDL provides for SOAP services.

10. API Testing and Evidence

Every endpoint should be tested with valid input. At least two invalid requests and one missing-resource request should also be tested. Returned status codes, JSON bodies, browser console output, and server terminal output must be checked. The boxes below are intentionally left as

placeholders because no final API-testing screenshots were included with the uploaded project files.

10.1 Minimum Test Matrix

Test	Case	Expected Result
GET /api/status	Valid	200; running message
GET /api/plans	Valid	200; array of plan objects
GET /api/plans/1	Valid	200; one plan
GET /api/plans/999	Missing resource	404; Plan not found
POST /api/registrations	Valid JSON body	201; created registration
POST /api/registrations	Missing required field	400; clear validation message
POST /api/registrations	Invalid tier	400; invalid tier message
GET /api/registrations/999	Missing resource	404; Registration not found
PUT /api/registrations/1	Valid complete body	200; replaced record
PATCH /api/registrations/1	Valid partial body	200; updated fields
DELETE /api/registrations/1	Valid ID	200; deleted record

10.2 Required Screenshots

Figure 1. Successful GET request

Sections

+

Q Search collections

ai code reviewer

Expensetracker pro

Fitness Application

GET status

home

Job Portal Project

Movies DB

Spring Boot

Web development express

New

Import

Collections

+

Q Search collections

ai code reviewer

Expensetracker pro

Fitness Application

GET status

home

Job Portal Project

Movies DB

Spring Boot

Fitness Application / status

Save

Share

GET

http://localhost:3000/api/status

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body

Cookies

Headers (7)

Test Results

200 OK

33 ms

269 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "message": "Lab 4 API is running"
3 }
```

Fitness Application / status

Save

Share

GET

http://localhost:3000/api/plans

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body

Cookies

Headers (7)

Test Results

200 OK

11 ms

876 B

Save Response

JSON

Preview

Visualize

```
1 {
2   {
3     "id": 1,
4     "name": "Beginner Plan",
5     "duration": "2 weeks",
6     "style": "Machine-assisted exercises",
7     "price": 49,
8     "level": "beginner",
9     "description": "Best for new members who want guided practice and safer pacing."
10  },
11  {
12    "id": 2,
13    "name": "Intermediate Plan",
14    "duration": "5 weeks",
15    "style": "Free weight exercises",
16    "price": 59,
17    "level": "intermediate",
18    "description": "Best for members who already know basic gym movements and want more structure."
19  },
20  {
21    "id": 3,
22    "name": "Advanced Plan",
23    "duration": "8 weeks",
24    "style": "Compound lifts + diet plans",
25    "price": 69,
26    "level": "advanced",
```

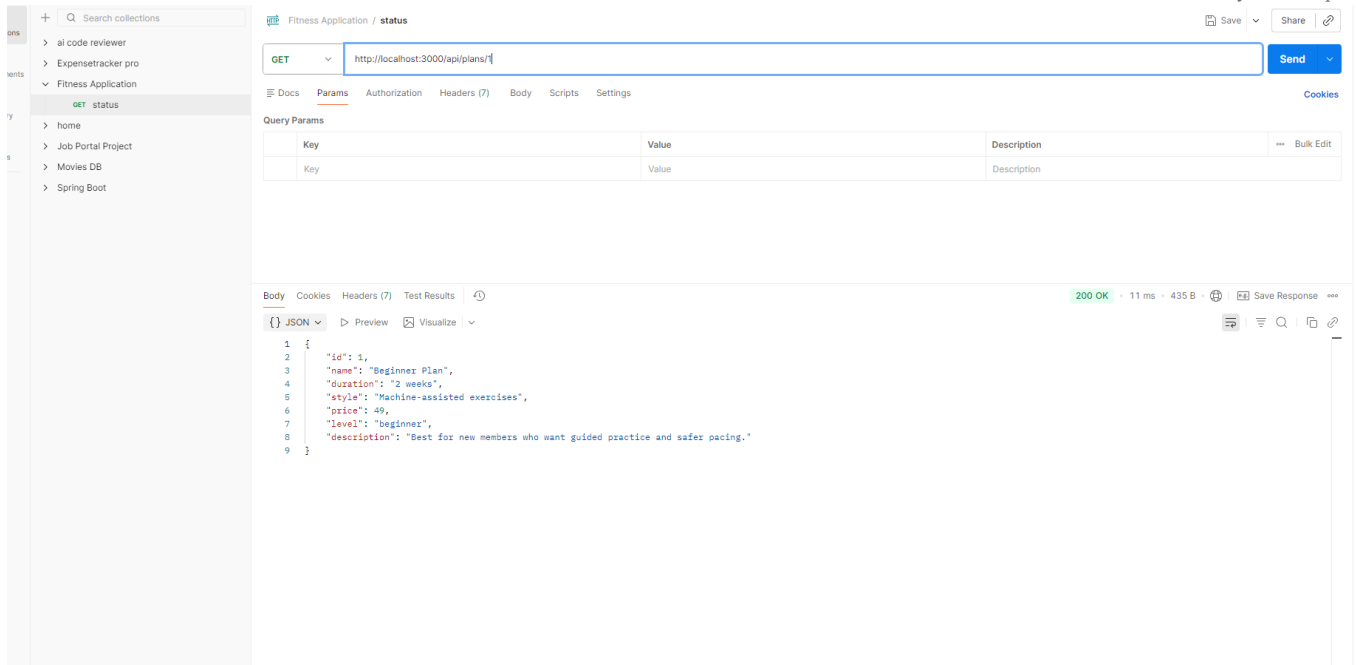


Figure 2. Successful POST request

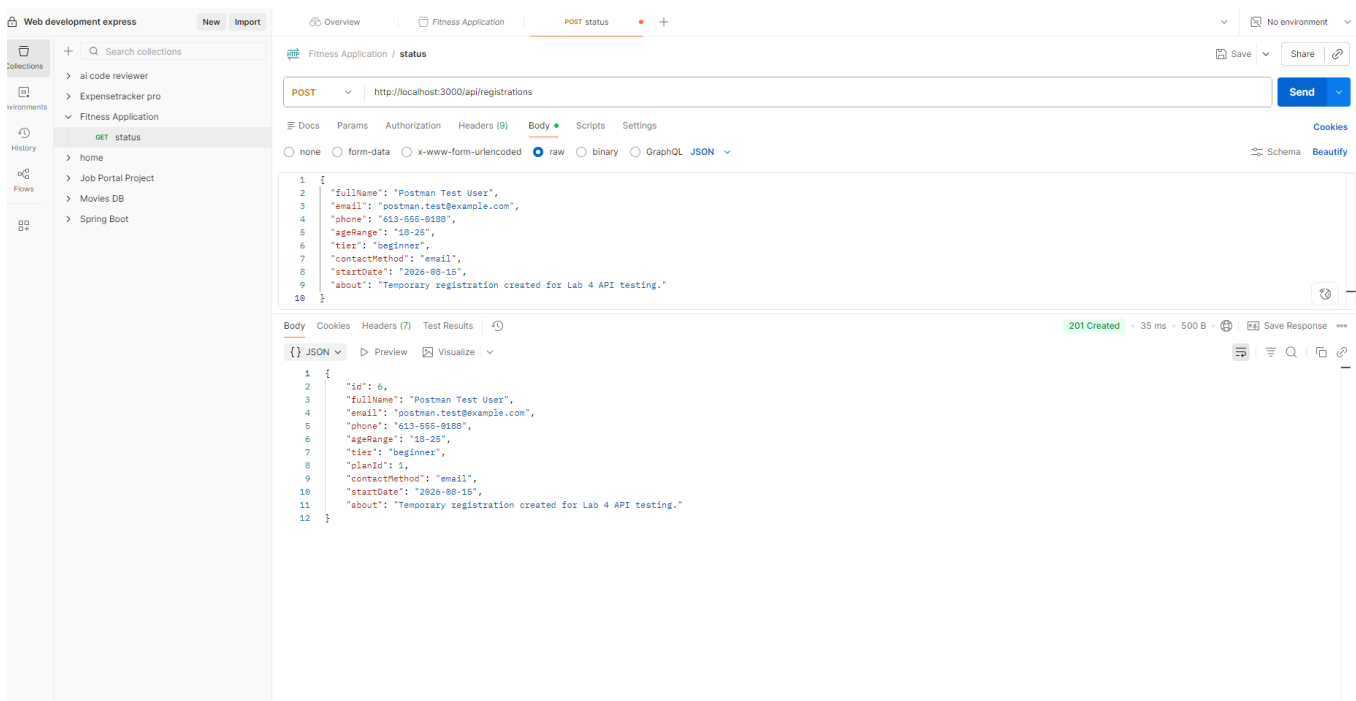


Figure 3. Invalid request

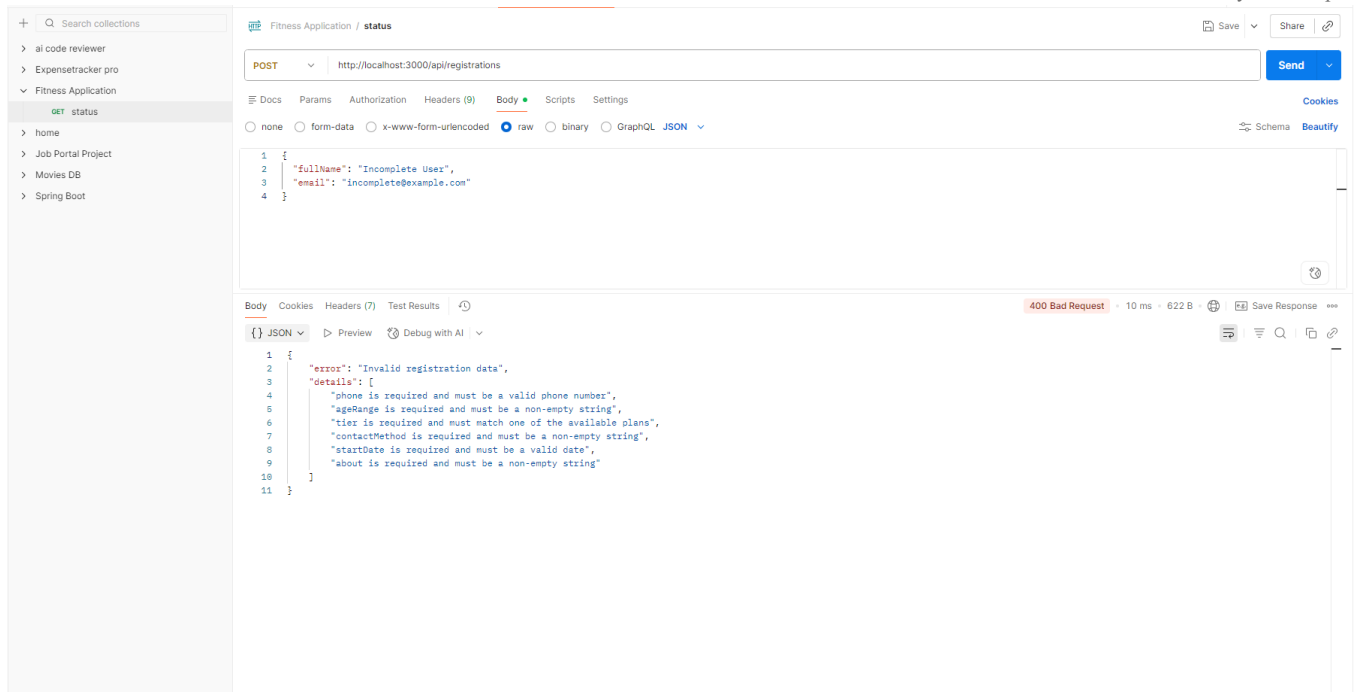


Figure 4. Missing resource

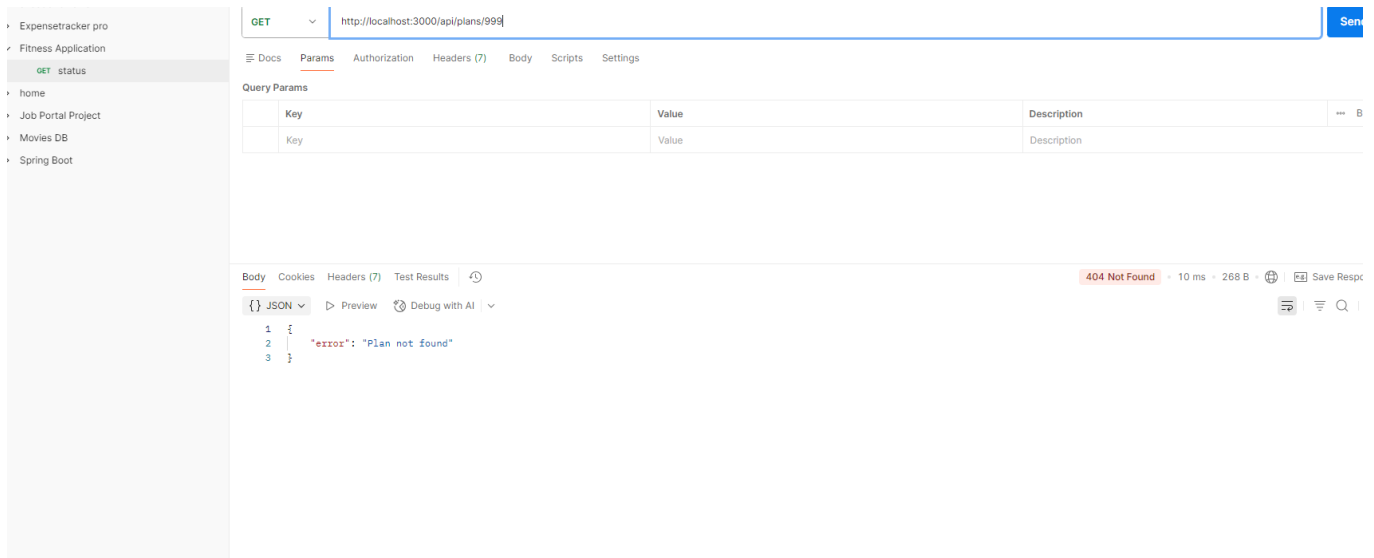


Figure 5. Front end using API data



```
localhost:3000/api/plans
pretty-print
{"id":1,"name":"Beginner Plan","duration":"2 weeks","style":"Machine-assisted exercises","price":49,"level":"beginner","description":"Best for new members who want guided practice and safer pacing."}, {"id":2,"name":"Intermediate Plan","duration":"5 weeks","style":"Free weight exercises","price":59,"level":"intermediate","description":"Best for members who already know basic gym movements and want more structure."}, {"id":3,"name":"Advanced Plan","duration":"8 weeks","style":"Compound lifts + diet plans","price":69,"level":"advanced","description":"Best for experienced members who want heavier training and more detailed planning."}]
```

11. Legacy Web Services: SOAP, WSDL, XML Schema, and JAX-RPC

11.1 SOAP

SOAP is a protocol for exchanging structured messages between systems. SOAP messages are normally XML documents contained in an envelope with a header and body. SOAP supports formal standards for faults, security, transactions, and reliable messaging, which made it common in large enterprise systems.

11.2 WSDL

WSDL, or Web Services Description Language, is an XML document that describes a SOAP service contract. It defines operations, message structures, data types, protocol bindings, and service locations. Tools can use WSDL to generate client proxy code.

11.3 XML Schema

XML Schema, commonly written as XSD, defines the structure and data types permitted in XML documents. A SOAP service can use XSD to define required elements, numbers, strings, dates, restrictions, and nested objects.

11.4 JAX-RPC

JAX-RPC was a Java API for building XML-based remote procedure call web services and clients. It allowed developers to call a remote SOAP operation in a style similar to invoking a Java method, often using generated stubs and service classes. Newer Java web-service APIs replaced it.

11.5 Why the Stack Is Considered Legacy

SOAP-based systems are more verbose and complex than the JSON-based APIs commonly used by browsers. WSDL and XML Schema provide strong contracts but increase tooling and configuration requirements. REST is usually simpler for modern CRUD applications. SOAP and WSDL are still used in long-lived banking, insurance, telecommunications, healthcare, and government systems where formal contracts, existing integrations, or enterprise security standards are important.

11.6 SOAP/WSDL Compared with REST/OpenAPI

Aspect	SOAP/WSDL	REST/OpenAPI
Message format	XML	Usually JSON
Contract	WSDL and XML Schema	OpenAPI and JSON Schema-style definitions
Design style	Operation-based	Resource-based
Typical interaction	Call named service operations	Use HTTP methods on resource URLs
Complexity	Higher and more formal	Usually lower and browser-friendly
Common use	Enterprise and legacy integrations	Public APIs and modern web applications

12. Modern API Comparison: REST, GraphQL, and gRPC

API Style	Main Format	Best Use Case	Main Strength
REST + OpenAPI	JSON over HTTP	Public and CRUD-style APIs	Simple, cacheable, and widely supported

API Style	Main Format	Best Use Case	Main Strength
GraphQL	Usually JSON over HTTP	Clients that need flexible combinations of related data	The client selects the exact fields it needs
gRPC	Protocol Buffers over HTTP/2	Internal service-to-service communication	Fast, compact, strongly typed contracts

12.1 Why REST Is Appropriate for This Project

REST is the best fit for Uranus Fitness because the application uses simple plan and registration resources with predictable CRUD operations. Browser `fetch()` supports REST directly, JSON is easy to process in JavaScript, and the endpoints can be documented clearly with OpenAPI. GraphQL would add unnecessary query and schema complexity for a small dataset. gRPC is designed mainly for strongly typed internal services and is not as convenient for direct browser form and table interactions.

13. Security and Production Considerations

13.1 Current Laboratory Protections

- Client-side validation provides immediate feedback.
- Server-side validation protects the API from direct invalid requests.
- Safe error responses avoid exposing raw internal stack traces.
- Resource IDs and tier values are checked before records are returned or saved.
- The same Express origin serves both the browser client and API.

13.2 Additional Production Security

- Authentication to identify users and administrators.
- Authorization so ordinary users cannot read, update, or delete every registration.
- HTTPS to encrypt personal information while it travels over the network.
- Stronger input validation and sanitization for every field.
- Rate limiting to reduce abuse and automated request flooding.

- Restricted CORS configuration if the front end and API are hosted on different origins.
- Secure storage of personal data and a real database instead of a plain JSON file.
- Logging, monitoring, backups, and environment variables for configuration and secrets.

13.3 Deployment Thinking

The Express application could be deployed to a Node.js hosting platform or cloud application service. The deployment would install dependencies, run `npm start`, expose the configured port, and serve both the public client and API. For a real deployment, the port should come from an environment variable, JSON file storage should be replaced with a managed database, HTTPS should be enabled, and authentication and authorization should protect registration data.

14. Client-Server and Modern Web Services Checklist

Checklist Item	Status
The project includes a clear client-server structure.	Complete
The server runs using Node.js and Express.js.	Complete
The server provides at least four REST API endpoints.	Complete
The API uses JSON request and response bodies.	Complete
The API uses appropriate HTTP methods.	Complete
The API uses appropriate HTTP status codes.	Complete
The API includes server-side validation.	Complete
The API returns clear error messages for invalid requests.	Complete
The client uses <code>fetch()</code> to call the API.	Complete
The client dynamically displays data received from the API.	Complete
The client includes a form that sends data to the API.	Complete
API testing screenshots are included.	Complete

Checklist Item	Status
The report includes OpenAPI-style documentation.	Complete
The report explains SOAP, WSDL, XML Schema, and JAX-RPC.	Complete
The report compares SOAP/WSDL with REST/OpenAPI.	Complete
The report compares REST, GraphQL, and gRPC.	Complete
The project was tested in a browser.	Complete
The browser console and server terminal were checked for errors.	Complete
A README file explains how to run the project.	Complete

15. Testing and Debugging Process

Final testing should begin by running `npm start` and opening the application through `http://localhost:3000` rather than opening the HTML files directly. Each endpoint should then be tested through Swagger UI or an API client. The Network and Console panels in browser developer tools should confirm that `fetch()` requests complete successfully and that no JavaScript errors occur. The server terminal should also be checked for file access, routing, or parsing errors.

- Verify that all static HTML, CSS, JavaScript, and image files load.
- Verify that the schedule table is filled from `GET /api/plans`.
- Test filtering, sorting, plan details, and `localStorage` plan selection.
- Submit valid and invalid registration forms.
- Confirm 200, 201, 400, and 404 status codes.
- Confirm that `registrations.json` changes after POST, PUT, PATCH, and DELETE.
- Compare Swagger documentation with the actual routes and response bodies.
- Test desktop, tablet, and mobile widths.
- Check keyboard focus states and accessible status messages.

16. Reflection

The main challenge was changing the Lab 3 schedule from a browser-only array into a true client-server feature without losing the existing filtering, sorting, and details behavior. This was solved by keeping the interface logic in `schedule.js` while changing the source of the plans array to `GET /api/plans`. This creates one source of truth in `plans.json` and avoids repeating the same plan records in HTML, JavaScript, and JSON.

A second challenge was handling asynchronous requests and failures. The solution was to use `async/await`, check `response.ok`, convert responses using `response.json()`, and use `try/catch` for network failures. Clear status messages were added so the user sees loading, success, validation, or API failure feedback instead of a silent empty page.

A third challenge was protecting data quality. Client-side validation improves usability, but it cannot be trusted as the only protection. The server therefore needs to repeat required-field and tier validation and return 400 or 404 responses. Separating routers, `dataStore.js`, JSON files, and OpenAPI documentation also made the code easier to explain and maintain.

The project demonstrates the full request-response cycle: the browser sends HTTP requests, Express routes them, the data layer reads or updates JSON records, the server returns JSON with a status code, and browser JavaScript updates the interface. REST and OpenAPI are the most suitable technologies for this small CRUD application because they are simple, widely supported, and easy to test.